Smart-ASHA: Software & Firmware Engineering Post-Mortem

This document records the software, firmware, and algorithmic challenges encountered during the development of the Smart-ASHA multi-modal diagnostic system on the ESP32-S3, along with their root causes and technical resolutions.

1. The Dual Edge Impulse Library Collision ("The Twin Problem")

The Challenge

When attempting to compile two Edge Impulse C++ SDK exports simultaneously—`Smart_ASHA_2.0_inferencing` (a 2D CNN trained on $24 \times 32 = 768$ pixel thermal heatmaps for Fever detection) and a second 1D sensor impulse—the Arduino IDE build failed with fatal linker collisions.

Root Cause

Edge Impulse C++ SDKs rely on a global, static runtime architecture. Both libraries exported:

- Identical global function prototypes: `EI_IMPULSE_ERROR run_classifier(...)`
- Identical static impulse descriptors: `const ei_impulse_t ei_default_impulse`
- Identical data structures: `ei_impulse_result_t`, `signal_t`
- Identical internal folder includes: `#include "model-parameters/model_variables.h"`

Renaming the outer headers or moving internal files into the root sketch folder broke relative compiler paths, throwing:

```
fatal error: model-parameters/model_variables.h: No such file or directory
```

Forcing the linker to resolve the duplicate symbols caused cross-model pointer hijacking: runtime calls dispatched inference using one model's metadata against another model's tensor parameters.

Engineering Resolution

1. **Model Consolidation:** Standardized firmware on a single Edge Impulse runtime (`Smart_ASHA_2.0_inferencing`) dedicated to the complex 768-pixel thermal vision task.
2. **Algorithmic Offloading:** Shifted secondary diagnostic endpoints (Cardiac Arrhythmia, Hypoxia, and Multi-spectral Anemia ratios) to deterministic mathematical signal processing, eliminating static runtime conflicts.

2. Guru Meditation: `LoadStoreAlignment` Panic (EXCCAUSE `0x00000009`)

Crash Log Signature

```
Guru Meditation Error: Core 1 panic'ed (LoadStoreAlignment). Exception was unha
EXCVADDR: 0x422d8f2a
Backtrace: 0x403855a6:0x3fcebb00 0x40384a72:0x3fcebb20 ...
Rebooting...
```

Root Cause

The Xtensa LX7 dual-core architecture inside the ESP32-S3 enforces strict 32-bit word alignment for SIMD and floating-point load/store instructions (`l32i` , `s32i`). Memory addresses accessed by these operations must be multiples of 4 bytes ($0x0, 0x4, 0x8, 0xC$).

The crash was triggered by two distinct issues:

1. **Buffer Shape & Dimension Mismatch:** The 2D CNN expected a full thermal frame buffer sized to `EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE` (768 elements). When an 18-element spectral array was passed into `numpy::signal_from_buffer` , the DSP pre-processing filter iterated past the 18th float into unallocated and unaligned memory addresses (`0x422d8f2a`), causing an immediate CPU exception.
2. **Tensor Arena Memory Alignment:** TFLite Micro vector optimizations require input buffers and tensor arenas to align to strict 16-byte or 32-byte boundaries.

Engineering Resolution

1. **Explicit Memory Alignment:** Declared all frame buffers and inference arrays with strict 32-byte memory alignment:

```
float thermalFrame[768] __attribute__((aligned(32)));
static float imageBuffer[768] __attribute__((aligned(32)));
```

2. **Safe Signal Dispatch:** Implemented strict shape validation prior to preprocessing, ensuring the 768-pixel vision model only executes against the MLX90640 buffer.

3. I2C Bus Driver Deadlock: `ESP_ERR_INVALID_STATE [259]`

Crash Log Signature

```
[ 34128][E][esp32-hal-i2c-ng.c:372] i2cWriteReadNonStop(): i2c_master_transmit_
[ 34139][E][Wire.cpp:520] requestFrom(): i2cWriteReadNonStop returned Error 259
```

Root Cause

1. **Multi-Core Bus Contention:** In dual-core configurations, Core 0 executed the AI task (`aiTask`) while Core 1 handled the UI rendering and sensor loops. Both cores called `tcaSelect()` simultaneously without synchronization, corrupting the internal state machine of the ESP32-S3 hardware I2C peripheral.
2. **Interrupt Latency During Inference:** Heavy floating-point calculations during CNN inference on Core 0 locked interrupts for durations longer than the I2C peripheral timeout threshold, causing the driver to enter an unrecoverable invalid state.

Engineering Resolution

1. **FreeRTOS Mutex Synchronization:** Protected all multiplexer switching and I2C operations behind a shared binary semaphore (`i2cMutex`):

```
if (xSemaphoreTake(i2cMutex, pdMS_TO_TICKS(200))) {
    tcaSelect(3);
    mlx.getFrame(thermalFrame);
    tcaDeselect();
    xSemaphoreGive(i2cMutex);
}
```

2. **Bus De-selection (`tcaDeselect`):** Before executing floating-point math or neural network inference, the firmware writes `0x00` to the TCA9548A register. This disconnects all sensor sub-buses, preventing floating I2C lines or noisy slaves from disrupting the controller.
3. **I2C Clock Throttling:** Lowered the primary I2C clock rate from 400 kHz to 50 kHz (`Wire.setClock(50000)`), expanding signal timing margins over jumper wires.

4. AS7343 API Method Mismatch: `setIntegrationTime`

Build Failure

```
error: 'class Adafruit_AS7343' has no member named 'setIntegrationTime'; did yo
```

Root Cause

Unlike simpler ambient light sensors, the Adafruit AS7343 library does not expose a consolidated `setIntegrationTime()` method. Integration time on the AS7343 hardware is controlled by two separate 8-bit registers:

- `ATIME` (number of integration cycles)
- `ASTEP` (integration step time in increments of $2.78\ \mu s$)

The total integration period is governed by the hardware equation:

$$t_{\text{int}} = (\text{ATIME} + 1) \times (\text{ASTEP} + 1) \times 2.78\ \mu\text{s}$$

Engineering Resolution

Replaced the non-existent function call with direct register configuration:

```
as7343.setGain(AS7343_GAIN_128X);
as7343.setATIME(59);   // 60 cycles
as7343.setASTEP(599);  // 600 steps -> 60 * 600 * 2.78us = 100.08 ms
```

5. Algorithmic Diagnostics (Bypassing Secondary AI)

To ensure high performance within 16 KB stack allocations, secondary diagnostics were converted to deterministic algorithms:

Cardiac Arrhythmia via HRV Variance

- Tracks consecutive R-wave peaks via hardware thresholding with a $250\ \text{ms}$ refractory period to avoid double-counting T-waves.
- Stores $N = 100$ RR-intervals and computes the standard deviation of normal-to-normal intervals (SDNN):

$$\mu_{RR} = \frac{1}{N}\sum_{i=1}^{N} RR_i, \quad \sigma_{RR} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(RR_i - \mu_{RR})^2}$$

- Flagged as Arrhythmia when $\sigma_{RR} > 110\ \text{ms}$.

Oxygen Saturation (SpO_2) Averaging

- Samples AC and DC components of Red ($660\ \text{nm}$) and Infrared ($880\ \text{nm}$) reflectance from the MAX30105.
- Accumulates instantaneous estimates over the 60-second scanning period:

$$R = \frac{\text{AC}_{\text{red}}/\text{DC}_{\text{red}}}{\text{AC}_{\text{ir}}/\text{DC}_{\text{ir}}}, \quad SpO_2 = 110 - 25 \times R$$

- Evaluates final averaged saturation against a clinical hypoxia threshold ($SpO_2 < 94\%$).